

The University of Nottingham Ningbo China

SCHOOL OF COMPUTER SCIENCE

A LEVEL 1 MODULE, AUTUMN SEMESTER 2023-2024

PROGRAMMING AND ALGORITHMS (COMP1038)

Time allowed: 1.0 hour (60 minutes)

(ANSWERS)

Bloom's Taxonomy

A subset of Bloom's Taxonomy when referring to the form of knowledge needed to answer an exam question. This employs the following three categories (using the original formulation of the terminology):

Knowledge – for what we often term 'bookwork' based upon recall of factual information'

Comprehension – or 'understanding', where students are asked to perform such actions as to describe, explain, classify ideas or concepts

Application – for the situations where a student is asked to undertake an 'unseen' task by using knowledge in a new way (or a variant upon one previously shown in class)

Question 1: What is the difference between call by value and call by reference in C programming? (2 marks)

Answer: (K) When using call by value, you are sending the value of a variable as parameter to a function, whereas call by reference sends the address of the variable. (1 mark) Also under call by value, the value of the caller function is not affected by the operations in the called function, while in the case of call by reference, values of caller can be affected by the called function. (1 mark)

Question 2: What does dynamic memory allocation mean? What are the functions in the C Standard Library that deal with dynamic memory allocation of variables? Write two simple C statements that allocate and release memory for an integer array that contains 1000 elements. (2 marks)

Answer: (CA) Dynamic memory allocation means that a program obtains and releases memory space (on the heap) at execution time rather than at compilation time. (0.5 mark)

malloc() and free() (0.5 mark)

```
int *ptr = malloc(1000*sizeof(int));
```

```
free(ptr);
```

(1 mark)

Question 3: Explain what would happen when the program exam.c, as shown in Figure 1, is compiled using the command as shown in Figure 2 on cslinux. (2 marks)



```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     double a = 5.0;
6
7     printf("%lf\n", sqrt(a));
8
9     return 0;
10 }
```

Figure 1: exam.c

```
[z2017233@CSLinux Desktop]$ cd exam_2023
[z2017233@CSLinux exam_2023]$ pwd
/home/nottingham.edu.cn/z2017233/Desktop/exam_2023
[z2017233@CSLinux exam_2023]$ ls
exam.c
[z2017233@CSLinux exam_2023]$ gcc exam.c -o exam
```

Figure 2: Compiling on cslinux

Answer: (A) The program will not compile because `-lm` is not included as one of the parameters when compiling the program. [1 mark]

The program will also have a warning, because `math.h` is not included in the code. [1 mark]

Question 4: Use a while loop to implement the same output as the following code. (2 marks)

```
for(a=1; a<=100; a++)
    printf("%d\n", a*a);
```

Answer: (A)

```
a=1;
while(a<=100){
    printf("%d\n", a*a);
    a++;
}
```

Question 5: There are several mistakes contained in the following C program. Identify and correct them in order to compile and run without error. (4 marks).

```
#include <stdio.h>

struct emp
{
    char n[20];
    int age;
};

int main(void)
{
    struct emp e1 = {"David", 23};
    struct emp e2;
    scanf("%s %d", e2.n, e2.age);
    if(structcmp(e1, e2) == 0)
        printf("Equal structs.");
```

```

        else
            printf("Unequal structs.");
    }

structcmp (struct emp x, struct emp y)
{
    if(x.n == y.n)
        if(x.age == y.age)
            return 0;
    return 1;
}

```

Answer: (CA)

The function structcmp should have a return type. `int structcmp(struct emp x, struct emp y)`

The function structcmp should have a declaration. `int structcmp (struct emp x, struct emp y);`

Two strings cannot be compared by using '=='. `if(x.n==y.n) => if (strcmp(x.n, y.n) == 0)`

Parameters of scanf() should be address or pointers. `scanf("%s %d", e2.n, e2.age); => scanf("%s %d", e2.n, &e2.age);`

[1 mark] for finding each mistake and correction.

Question 6: Complete the function body of *check* in the program below. This function should take two arguments, a variable of type integer called *input* and a pointer called *ptr* that points to an integer. A calculation to determine whether the *input* is odd, even or zero must be done within this function. If *input* is zero, the program should print 0. If *input* is odd, the program prints -1. If *input* is even, the program should print 1. You may assume that the user will only enter a single whole number in each prompt (i.e., no text, no floating-point numbers, no empty input, etc.) (3 marks)

```
#include <stdio.h>
```

```
void check(int input, int *ptr);
```

```
int main(void)
{
    int in = 0;
    int out = 0;

    while(in != -9999)
    {
        scanf("%d", &in);
    }
}

```

```

        check(in, &out);
        printf("%d\n", out);
    }
    return 0;
}

```

Answer: (CA)

```

void check(int input, int *ptr)
{
    if(input == 0)
        *ptr = 0;
    else if((input % 2) == 0)
        *ptr = 1;
    else
        *ptr = -1;
}

```

[1 mark] for correct function definition, [2 marks] for the if-else control structure in function body.

Question 7: An incomplete C program that employs a singly linked list is given below. Complete the printList function which prints out all values in the linked list. Each value should be printed in a line. (3 marks)

```

struct node{
    int data;
    struct node *next;
};
typedef struct node Node;

Node *head;

void InsertNodeAtFront(int data);
void printList();

void InsertNodeAtFront(int data)
{
    Node *temp = new Node;
    temp->data = data;
    temp->next = head;
    head = temp;
}

```

Answer: (CA)

```

void printList()
{
    Node *tmp = head;
    while(tmp != NULL)
    {
        printf("%d\n", tmp->data);
        tmp = tmp-> next;
    }
}

```

[1 mark] for defining the tmp node, [2 marks] for the iteration to print each item in the list.

Question 8: Identify potential mistakes contained in the function ‘func’ and suggest how to correct them. (3 marks)

```

#include <stdio.h>
#include <stdlib.h>
struct ex
{
    int i;
    float j;
    char *s;
};

void func (void)
{
    struct ex *p = malloc(sizeof(struct ex));
    p->s = malloc(20 * sizeof(char));
    scanf("%s", &p->s);
    printf("%s", p->s)
    free(p);
}

```

Answer: (CA)

&p->s in the line scanf("%s", &p->s); should be p->s [1 mark]

There is potential memory leak because allocated memory does not freed. Add free(p->s); before free(p); [2 marks]

Question 9: What is the output of the following program? (4 marks)

```
#include <stdio.h>

int f1( int x, int y)
{
    x=x+2;
    y=y+3;
    return x+y;
}

int f2( int *x, int y)
{
    *x=*x+2;
    y=y+3;
    return *x+y;
}

int f3 ( int *x, int *y)
{
    *x=*x+2;
    *y=*y+3;
    return *x+*y;
}

int f4( int x, int *y, int *z)
{
    x=x+*y;
    *y=*z+3;
    z=&x;
    *z=*y*2;
    return *z;
}

int main(int argc, char *argv[])
{
    int k=1, m=3, r=2;
    printf("1) %d %d %d \n", k, m, r);
    r=f1(k, m);
```

```
    printf("2) %d %d %d \n", k, m, r);  
    r=f2(&k, m);  
    printf("3) %d %d %d \n", k, m, r);  
    r=f3(&k, &m);  
    printf("4) %d %d %d \n", k, m, r);  
    r=f4(k, &m, &r);  
    printf("5) %d %d %d \n", k, m, r);  
    return 0;  
}
```

Answer: (CA)

- 1) 1 3 2
- 2) 1 3 9
- 3) 3 3 9
- 4) 5 6 11
- 5) 5 14 28

[1 mark] for each line of 2-5.